Intelligent Robotics (ASY2) · Lab 1

Kinematics and odometry

A differential drive and a mecanum drive as ROS 2 nodes

Duration	180 minutes	Work in	pairs
Script	Cycle 1, sections 1.1 and 1.2	Files	`ohm-odometry/student/`

1 Overview

You write two ROS 2 nodes for a simulated robot, one for a differential drive and one for a mecanum drive. Each node does three things:

- it turns the keyboard command into wheel speeds (inverse kinematics),
- it turns the measured wheel speeds back into the speed of the robot (forward kinematics),
- it adds up these speeds to a position (odometry).

The node talks to the simulator only through ROS topics. Which topics these are, you find out yourself.

Table 1: Schedule.

min	task	done when
0–20	section 3 Explore the simulator	table 2 is filled in
20–55	section 4 Differential drive: kinematics	the robot drives with the keys
55–85	section 5 Differential drive: odometry	the green outline follows the robot
85–95	break	
95–130	section 6 Mecanum drive: kinematics	the robot drives sideways with the keys
130–155	section 7 Mecanum drive: odometry	the green outline follows the robot
155–180	section 8 Extra tasks	

Notation as in the script: pose $\mathbf{p} = (x, y, \phi)^\mathsf{T}$, body twist $\xi = (v_x, v_y, \omega)^\mathsf{T}$. The x-axis points forward, y to the **left**, and the heading ϕ counts counter-clockwise. Speeds are in $\mathrm{m/s}$ and $\mathrm{rad/s}$, wheel speeds in $\mathrm{rad/s}$. The equations you need are repeated in the tasks, with the numbers they have in the script.

2 Getting started

Your own copy. Fork `ohm-odometry` on GitHub (button *Fork* at the top right), then clone your fork and the simulator into the `src/` folder of your workspace:

```
git clone git@github.com:<your-name>/ohm-odometry.git
```

```
git clone git@github.com:christianpfitzner/ir-mecanum-lab.git
```

Your work then lives in your own repository. Commit after every task and push, so nothing is lost at the end of the session.

Build. Build both once, then source the workspace in every new terminal:

```
colcon build --symlink-install --packages-select mecanum_lab_interfaces mecanum_lab
    ohm_odometry
```

```
source install/setup.bash
```

You do not need to build again after a change to your node. Just start it again.

Two ways to start the simulator.

- `diff_demo.launch.py` and `mecanum_demo.launch.py`: the simulator uses its own kinematics. The keys drive the robot, no node of yours is needed.
- `diff.launch.py` and `mecanum.launch.py`: the robot moves only on the wheel speeds your node sends. Without your node it stands still, with wrong kinematics it drives wrong.

Your files. `student/diff_drive_node.py` for tasks 1 and 2, `student/mecanum_drive_node.py` for tasks 3 and 4. Each is a complete ROS 2 node. You only change the lines marked `TODO`; subscribing, publishing and reading the messages are already there. Your node runs in a second terminal:

```
ros2 run ohm_odometry diff_drive_node.py
```

What the node does. 50 times per second the simulator sends the wheel encoders. For each message the node computes the wheel speeds, calls your forward kinematics and your odometry step, and publishes the new pose. The simulator draws it as a green outline of the robot, with a line for the heading and a green line for the path. Also 50 times per second, the node takes the last keyboard command, calls your inverse kinematics and publishes the wheel speeds.

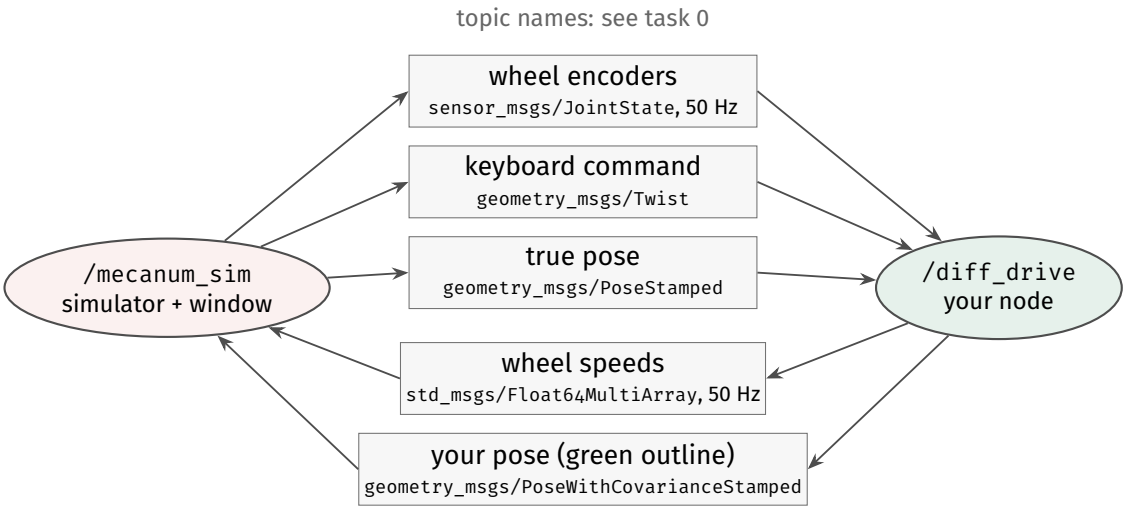


Figure 1: The simulator and your node, drawn like `rqt_graph`: nodes are ellipses, topics are boxes, arrows point from publisher to subscriber.

3 Task 0: explore the simulator

20 min

1. Start the simulator with its own kinematics:

   ```
   ros2 launch ohm_odometry diff_demo.launch.py
   ```

   Click into the window and drive with `w/s` (forward, back) and `q/e` (turn). What happens with `a/d` (sideways)?

2. In a second terminal, list the topics and look at each one in table 2:

   ```
   ros2 topic list
   ```

   ```
   ros2 topic info -v <topic>
   ```

3. Drive forward and watch the encoder message. Which numbers change? How often does it arrive?

   ```
   ros2 topic echo <topic>
   ```

   ```
   ros2 topic hz <topic>
   ```

4. Open `rqt_graph` and switch off the filters *Dead sinks* and *Leaf topics*. Compare with fig. 1. What is still missing?

5. Stop the simulator (Ctrl-C) and start `mecanum_demo.launch.py`. What changes in the encoder message?

> The last two topics have no publisher yet, the simulator only listens to them. They still show up in `ros2 topic list`. Your pose goes to the topic of type `PoseWithCovarianceStamped`.

Table 2: The topics between your node and the simulator. Fill in.

	topic name	message type	rate
wheel encoders			
keyboard command			
true pose of the robot			
wheel speeds (your node)			
pose of your odometry			

4 Task 1: differential drive, kinematics

35 min

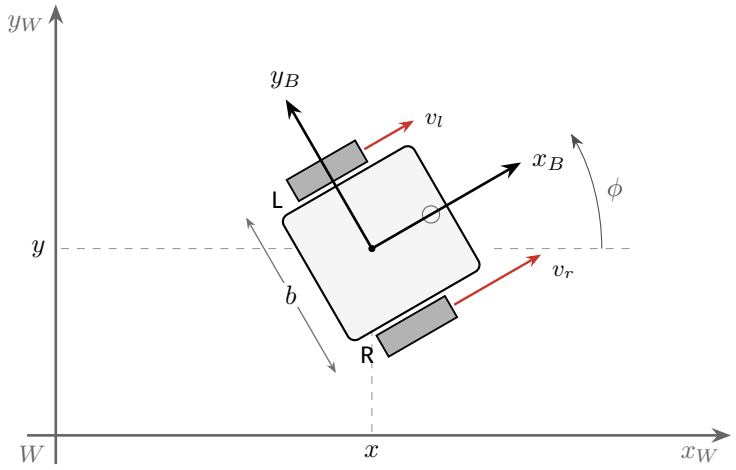


Figure 2: Differential drive at the pose (x, y, ϕ). The wheels sit $b/2$ left and right of the centre. Compare figure 1.4 of the script.

Stop the demo and start the simulator for your own node. The robot no longer reacts to the keys.

```
ros2 launch ohm_odometry diff.launch.py
```

Lecture script, section 1.2.2

With the speeds v_l, v_r at which the left and the right wheel roll, and the track b:

$$v = \frac{v_r + v_l}{2}, \qquad \omega = \frac{v_r - v_l}{b} \qquad \Rightarrow \qquad v_l = v - \omega\,\frac{b}{2}, \qquad v_r = v + \omega\,\frac{b}{2}. \tag{1.30}$$

TODO 1 Write the topic names of table 2 into the top of `student/diff_drive_node.py`.

TODO 2: inverse kinematics `inverse_kinematics(v, omega)` with (1.30). It gives v_l, v_r in $\mathrm{m/s}$, the simulator wants the wheels to turn at w in $\mathrm{rad/s}$. A wheel with radius r_w that rolls at v turns at $w = v/r_w$. In the code, b is `B` and r_w is `R`.

Start your node and drive. Forward: both wheels turn forward at the same speed. `q` (turn left): the left wheel turns backward, the right one forward. `ros2 topic echo` on your wheel-speed topic shows the numbers.

TODO 3: forward kinematics `forward_kinematics(w_left, w_right)`: the left half of (1.30). Turn the wheel speeds back into rolling speeds first.

Checkpoint The robot drives forward with `w` and turns left with `q`. `rqt_graph` looks like fig. 1.

5 Task 2: differential drive, odometry

30 min

Each encoder message is one step of Δt seconds. Your forward kinematics gives v and ω for it; the odometry adds up the steps to a pose.

Lecture script, section 1.2.2

Distance of the wheels and of the kinematic centre:

$$\Delta s_l = v_l\,\Delta t, \qquad \Delta s_r = v_r\,\Delta t, \qquad \Delta s_k = \frac{\Delta s_l + \Delta s_r}{2}. \tag{1.27}$$

Radius of the arc and change of the heading:

$$r = \frac{b}{2}\cdot\frac{\Delta s_r + \Delta s_l}{\Delta s_r - \Delta s_l}, \qquad \Delta\phi = \frac{\Delta s_r - \Delta s_l}{b}. \tag{1.28}$$

Pose update, with ϕ_0 the heading before the step:

$$\Delta s = 2r\sin\Big(\frac{\Delta\phi}{2}\Big), \qquad \Delta x = \Delta s\cos\Big(\frac{\Delta\phi}{2}+\phi_0\Big), \qquad \Delta y = \Delta s\sin\Big(\frac{\Delta\phi}{2}+\phi_0\Big). \tag{1.29}$$

TODO 4: odometry step `odometry_step(x, y, phi, v, omega, dt)` with (1.27) to (1.29). In the function you have v and ω, so $\Delta s_k = v\,\Delta t$, $\Delta\phi = \omega\,\Delta t$ and $r = \Delta s_k/\Delta\phi$.

When the robot drives straight, $\Delta\phi = 0$ and the radius is infinite: Python divides by zero. What is Δs in that case? Handle it with an `if`.

Start your node and drive. Press `t` in the window to show the true path of the robot next to the green path of your odometry. While the robot stands still, compare the two poses:

```
ros2 topic echo <true pose topic> --once
```

```
ros2 topic echo <your pose topic> --once
```

Checkpoint After a minute of driving in curves and straight lines, the green outline still covers the robot.

6 Task 3: mecanum drive, kinematics

35 min

Stop the simulator and your node, then start both for the mecanum robot (the demo is `mecanum_demo.launch.py`):

```
ros2 launch ohm_odometry mecanum.launch.py
```

```
ros2 run ohm_odometry mecanum_drive_node.py
```

TODO 1, the topic names, is the same as in task 1.

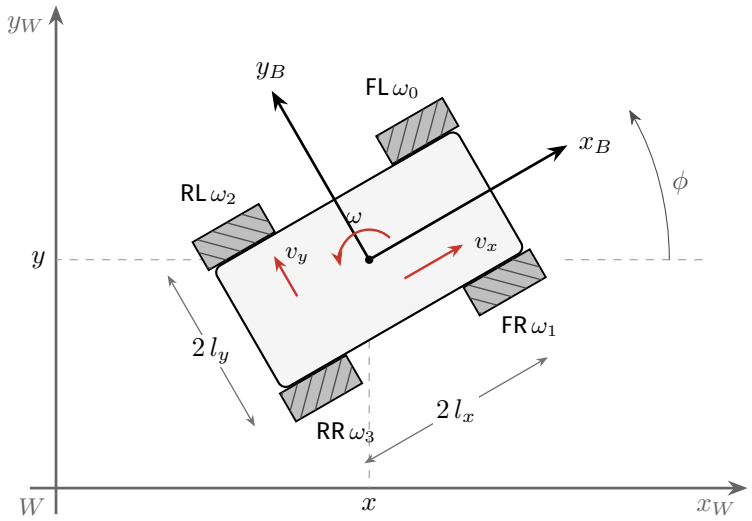


Figure 3: Mecanum drive at the pose (x, y, ϕ), rollers in X layout. Compare figure 1.11 of the script.

Lecture script, section 1.2.6

Wheels ω_0 front left, ω_1 front right, ω_2 rear left, ω_3 rear right, wheel radius r, half distances l_x, l_y (R, LX, LY in the code).

$$\begin{pmatrix}\omega_0\\ \omega_1\\ \omega_2\\ \omega_3\end{pmatrix} = \frac{1}{r}\begin{pmatrix}1 & -1 & -(l_x+l_y)\\ -1 & -1 & -(l_x+l_y)\\ 1 & 1 & -(l_x+l_y)\\ -1 & 1 & -(l_x+l_y)\end{pmatrix}\begin{pmatrix}v_x\\ v_y\\ \omega\end{pmatrix} \tag{1.39}$$

$$\begin{pmatrix}v_x\\ v_y\\ \omega\end{pmatrix} = \frac{r}{4}\begin{pmatrix}1 & -1 & 1 & -1\\ -1 & -1 & 1 & 1\\ \frac{-1}{l_x+l_y} & \frac{-1}{l_x+l_y} & \frac{-1}{l_x+l_y} & \frac{-1}{l_x+l_y}\end{pmatrix}\begin{pmatrix}\omega_0\\ \omega_1\\ \omega_2\\ \omega_3\end{pmatrix} \tag{1.40}$$

The right wheels count the other way round. In the script, ω_1 and ω_3 are negative when the robot drives forward, as on a robot whose right motors are mounted mirrored. In the simulator every wheel is positive when it rolls forward. So $w_{\mathrm{FR}} = -\omega_1$ and $w_{\mathrm{RR}} = -\omega_3$.

TODO 2: inverse kinematics `inverse_kinematics(vx, vy, omega)` with (1.39) and the note above. The line for the front left wheel is already there.

TODO 3: forward kinematics `forward_kinematics(w_fl, w_fr, w_rl, w_rr)` with (1.40). Use $\omega_1 = -w_{\mathrm{FR}}$ and $\omega_3 = -w_{\mathrm{RR}}$.

Checkpoint Forward, sideways (`a`/`d`) and turning each work without the other two motions.

7 Task 4: mecanum drive, odometry

25 min

The mecanum robot also moves sideways. Its displacement $(v_x\,\Delta t,\ v_y\,\Delta t)$ is given in the frame of the robot and has to be rotated into the frame of the hall with $\mathbf{R}(\phi)$, the rotation matrix (1.6) of the script. Taking the heading in the middle of the step, $\bar\phi = \phi_k + \Delta\phi/2$ with $\Delta\phi = \omega\,\Delta t$, as in (1.29):

$$\begin{pmatrix} x \\ y \end{pmatrix}_{k+1} = \begin{pmatrix} x \\ y \end{pmatrix}_{k} + \mathbf{R}(\bar\phi) \begin{pmatrix} v_x \\ v_y \end{pmatrix} \Delta t, \qquad \phi_{k+1} = \phi_k + \Delta\phi, \qquad \mathbf{R}(\phi) = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}. \tag{1}$$

For $v_y = 0$ this is the step of task 2. (The exact step along the arc is shorter by the factor $\sin(\Delta\phi/2)/(\Delta\phi/2)$. At 50 steps per second that factor differs from 1 by less than 10^{-4}.)

TODO 4: odometry step `odometry_step(x, y, phi, vx, vy, omega, dt)`: multiply out eq. (1) and write the three lines for x, y and ϕ.

Checkpoint Drive forward, sideways and in curves. The green outline still covers the robot.

8 Extra tasks

Both tasks drive the robot with your own odometry instead of the keyboard. Each has its own node, `student/square_node.py` and `student/goto_node.py`. They take the topic names, the kinematics and the odometry of your drive node from tasks 1 to 4 and only replace the keyboard by a controller in `command()`; `self.pose` holds (x, y, ϕ) of your odometry. Set `ROBOT` at the top of the file to `"diff"` or `"mecanum"`. Use $\mathrm{wrap}(\alpha) = \mathrm{atan2}(\sin\alpha, \cos\alpha)$ to keep angles in $(-\pi, \pi]$.

Start the matching simulator and the node instead of your drive node, for example:

```
ros2 run ohm_odometry square_node.py
```

E1: drive a square by odometry

TODO E1a and E1b in `square_node.py`. Side length $a = 1\,\mathrm{m}$. Store the pose (x_s, y_s, ϕ_s) at the start of every side and every corner.

- **Differential drive.** Side: $v = 0.2\,\mathrm{m/s}$, $\omega = 0$, until

$$d = \sqrt{(x - x_s)^2 + (y - y_s)^2} \;\geq\; a. \tag{2}$$

  Corner: $v = 0$, $\omega = 0.5\,\mathrm{rad/s}$, until $|\mathrm{wrap}(\phi - \phi_s)| \geq \pi/2$. Four sides, four corners.

- **Mecanum drive.** The robot needs no corners. Drive the four sides with

$$(v_x, v_y) = (0.2, 0), \quad (0, 0.2), \quad (-0.2, 0), \quad (0, -0.2)\ \mathrm{m/s},$$

  each until eq. (2) holds.

How far from its start does the robot end, measured with the true pose? Where does the error come from?

E2: drive to a point

TODO E2a to E2c in `goto_node.py`. Goal $\mathbf{g} = (x_g, y_g)$, here 2 m in front of and 1 m to the left of the start. Error in the hall:

$$\Delta x = x_g - x, \qquad \Delta y = y_g - y, \qquad \rho = \sqrt{\Delta x^2 + \Delta y^2}. \tag{3}$$

Stop when $\rho < 0.05\,\mathrm{m}$.

- **Differential drive.** Angle between the heading and the direction to the goal:

$$\alpha = \mathrm{wrap}\big(\,\mathrm{atan2}(\Delta y, \Delta x) - \phi\big), \qquad v = k_\rho\,\rho\cos\alpha, \qquad \omega = k_\alpha\,\alpha, \tag{4}$$

  with $k_\rho = 0.5\,\mathrm{s}^{-1}$, $k_\alpha = 1.5\,\mathrm{s}^{-1}$ and v limited to $0.3\,\mathrm{m/s}$. The factor $\cos\alpha$ slows the robot down while it still points away from the goal.

- **Mecanum drive.** Rotate the error into the frame of the robot with $\mathbf{R}^{-1} = \mathbf{R}^{\mathsf{T}}$ (script 1.1.6) and drive straight at the goal:

$$\begin{pmatrix} e_x \\ e_y \end{pmatrix} = \mathbf{R}(\phi)^{\mathsf{T}} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}, \qquad v_x = k_\rho\, e_x, \qquad v_y = k_\rho\, e_y, \qquad \omega = 0. \tag{5}$$

  Limit $\sqrt{v_x^2 + v_y^2}$ to $0.3\,\mathrm{m/s}$ by scaling both components.

Compare the paths of both robots to the same goal.